Compressed Representations of Set Collections

Laurea Magistrale in Informatica

Luca Lombardo

29 May 2026

Dipartimento di Informatica
Università di Pisa

Set collections

Definition (Set collection)

A set collection over a finite, totally ordered universe U is a family

$$\mathcal{S} = \{S_1, \ldots, S_m\}, \quad S_i \subseteq U.$$

We write $u = |U|$ and $n = \sum_{i=1}^{m} |S_i|$, and assume $U = [1..u]$.

The encoded object is the whole collection, but each query selects one set S_i and asks for ordered-set information inside it.

- $\mathbf{member}(S_i, x)$: whether $x \in S_i$.
- $\mathbf{access}(S_i, j)$: the j-th element of S_i.
- $\mathbf{rank}(S_i, x)$: the number of elements of S_i at most x.
- $\mathbf{predecessor}(S_i, x)$: the largest element of S_i at most x.
- $\mathbf{successor}(S_i, x)$: the smallest element of S_i at least x.

Counting is not querying

A representation identifies one object among many possible objects. A bit is redundant when changing it does not change which object is identified.

Proposition (Counting lower bound)

Let $\mathcal{F}$ be a finite class of objects. If every object in $\mathcal{F}$ is represented by a binary codeword of length at most L, then

$$2^L \geq |\mathcal{F}| \quad \text{and therefore} \quad L \geq \lceil \log |\mathcal{F}| \rceil.$$

The bound is only a **space target**. A shortest description may identify the object, but it may give no direct way to test membership, count elements, or select the next one.

We seek representations close to the counting cost that still answer queries.

Independent encoding

Per set. If relations inside $\mathcal{S}$ are ignored, each set is chosen as a subset of the full universe.

$$S_i \subseteq U, \quad |S_i| \ \text{fixed} \quad \Longrightarrow \quad \binom{u}{|S_i|} \ \text{choices.}$$

Whole collection. Describing the sets independently adds the per-set costs, giving the baseline space for the collection.

$$H_{\text{wc}}(\mathcal{S}) = \sum_{i=1}^{m} \lg \binom{u}{|S_i|}$$

Independent encoding pays for each set in isolation, as if the other sets were not present. Any smaller description must use redundancy across the collection.

Containment references

A first way to use redundancy is to describe a set inside a containing reference.

Definition (Containment step)

$$Q \subseteq P \quad \Longrightarrow \quad \mathrm{cost} = \lg \binom{|P|}{|Q|} \leq \lg \binom{u}{|Q|}.$$

Alanko et al. (2025) build a **containment hierarchy** where each set is represented inside a smallest proper superset, or inside U if no such set exists.

$$U \to \ldots \to P \to Q, \quad Q = P \setminus D.$$

This is query-friendly because every parent-to-child step is a deletion. It becomes compressive when the collection contains a **deep nesting** structure.

Symmetric differences

If neither $A \subseteq B$ nor $B \subseteq A$, containment cannot relate the two sets directly, even when they almost coincide.

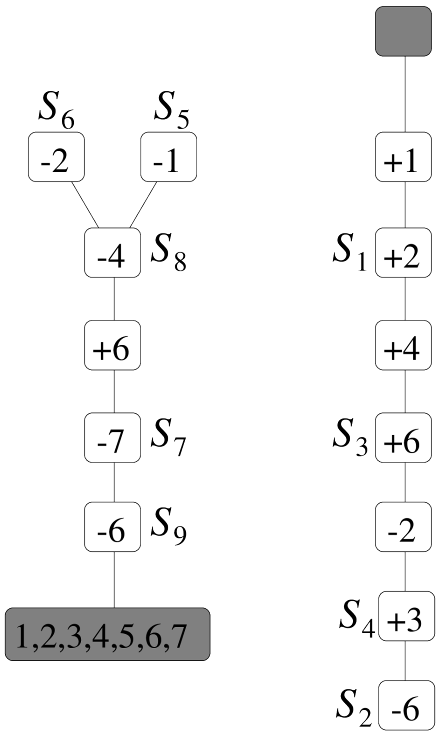
Definition (Symmetric difference)

$$A \triangle B = (A \setminus B) \cup (B \setminus A), \quad |A \triangle B| = |A \setminus B| + |B \setminus A|$$

If S is described from a chosen reference R, only the changed elements are stored as signed edits.

$$+x \quad \text{for} \quad x \in S \setminus R, \quad -y \quad \text{for} \quad y \in R \setminus S$$

Each reference costs $|S \triangle R|$. Gagie, He, and Navarro (2026) choose the references for the whole collection, then store the resulting signed edits on two rooted trees, called **indel trees**.



Choosing references

To query a set through a path, we first choose a reference for it. That choice fixes what the path records: deletions for containment, signed edits for symmetric difference.

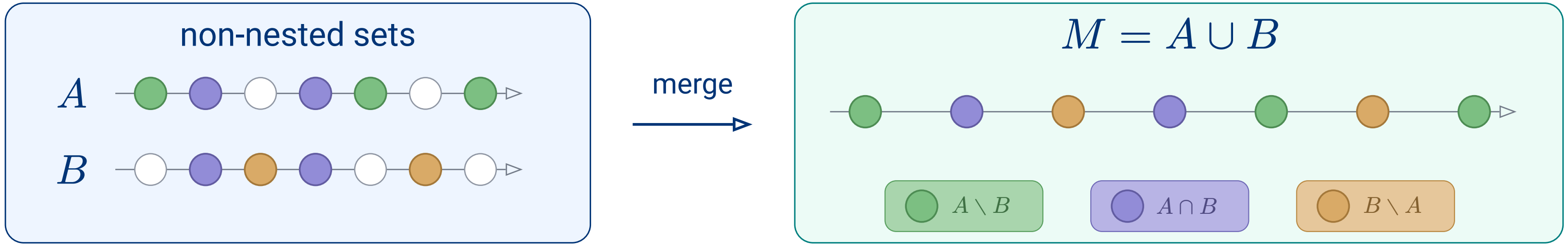
Structure	Reference domain	Stored differences
Containment	containing set $P \supseteq S$	deletions $P \setminus S$
Symmetric difference	nearby existing set R	signed edits $S \triangle R$

$$\text{available references} \to \text{stored differences} \to \text{queryable paths}$$

Keep deletion paths, but allow new containing references.

If two sets overlap but neither contains the other, the pair itself gives no containment reference. The smallest containing reference we can add is their union.

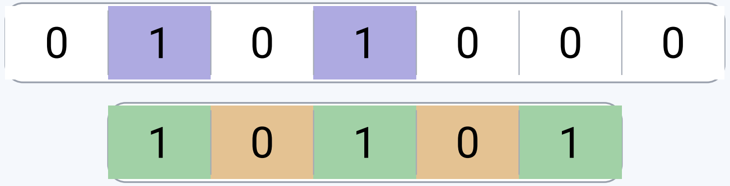
Union as reference



Encode A and B from M

First bitvector: marks $A \cap B$ inside M

Second bitvector: on $M \setminus (A \cap B)$, 1 marks $A \setminus B$

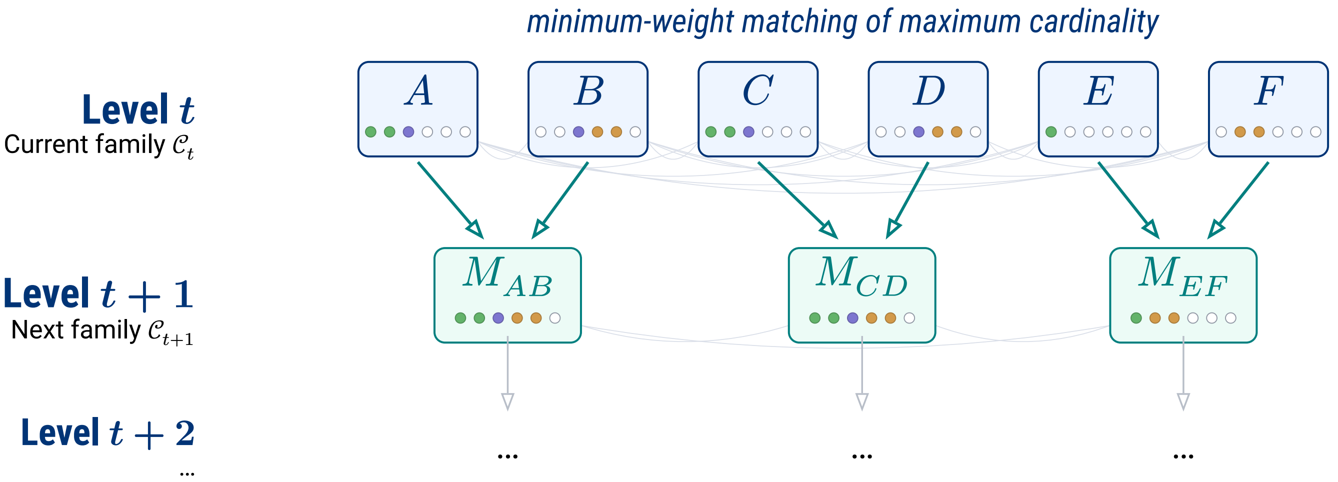


Once M is stored, (A, B) is determined by how the positions of M split into three regions.

$$w(A,B) = \underbrace{\lg\binom{|M|}{k,l,r}}_{\text{choose the three regions}} + \underbrace{O(\lg|M|)}_{\text{write region sizes}} \quad \text{where} \quad k = |A \cap B|, \quad l = |A \setminus B|, \quad r = |B \setminus A|$$

Set-Union Matching (SUM)

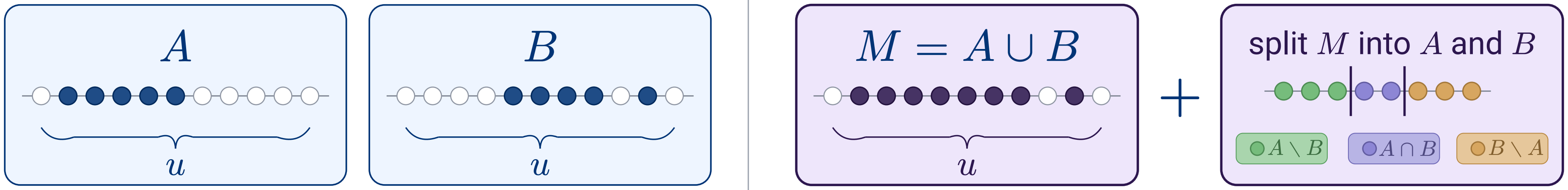
The construction builds a **bottom-up hierarchy** from the input sets: each round scores every pair in the current family and replaces the selected pairs by their unions.



We need a **score** for each candidate merge, so that each round can choose as many pairs as possible without using the same current set twice and with minimum total cost.

Merge score

Old description | **New description**



store A

$$\lg\binom{u}{|A|}$$

store B

$$\lg\binom{u}{|B|}$$

store the union

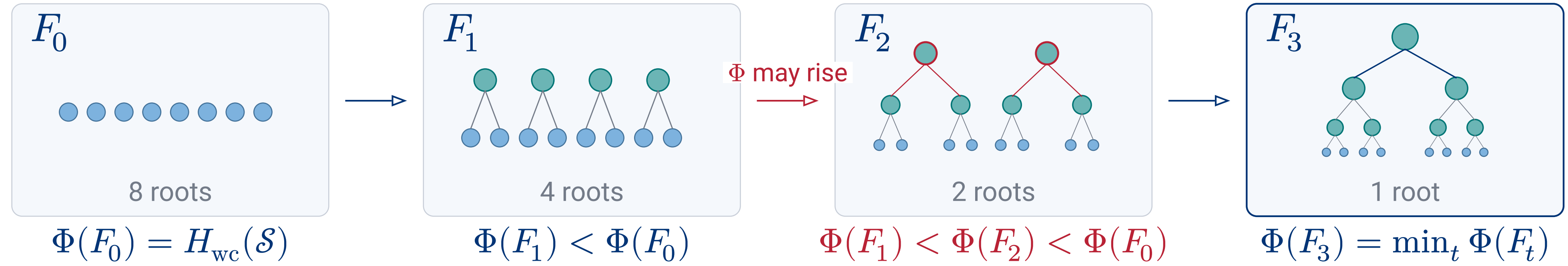
$$\lg\binom{u}{|M|}$$

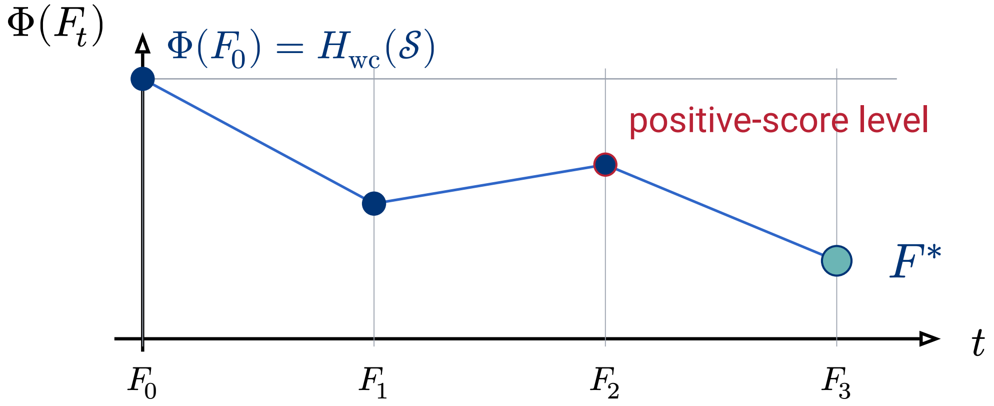
$$w(A,B)$$

$$\Delta\Phi(A,B) = \underbrace{\lg\binom{u}{|M|}}_{\text{store the union}} + \underbrace{w(A,B)}_{\text{split into } A \text{ and } B} - \underbrace{\lg\binom{u}{|A|}}_{\text{old support of } A} - \underbrace{\lg\binom{u}{|B|}}_{\text{old support of } B}.$$

$\Delta\Phi < 0$: saves bits $\Delta\Phi = 0$: same counted cost $\Delta\Phi > 0$: not locally profitable

Greedy matching

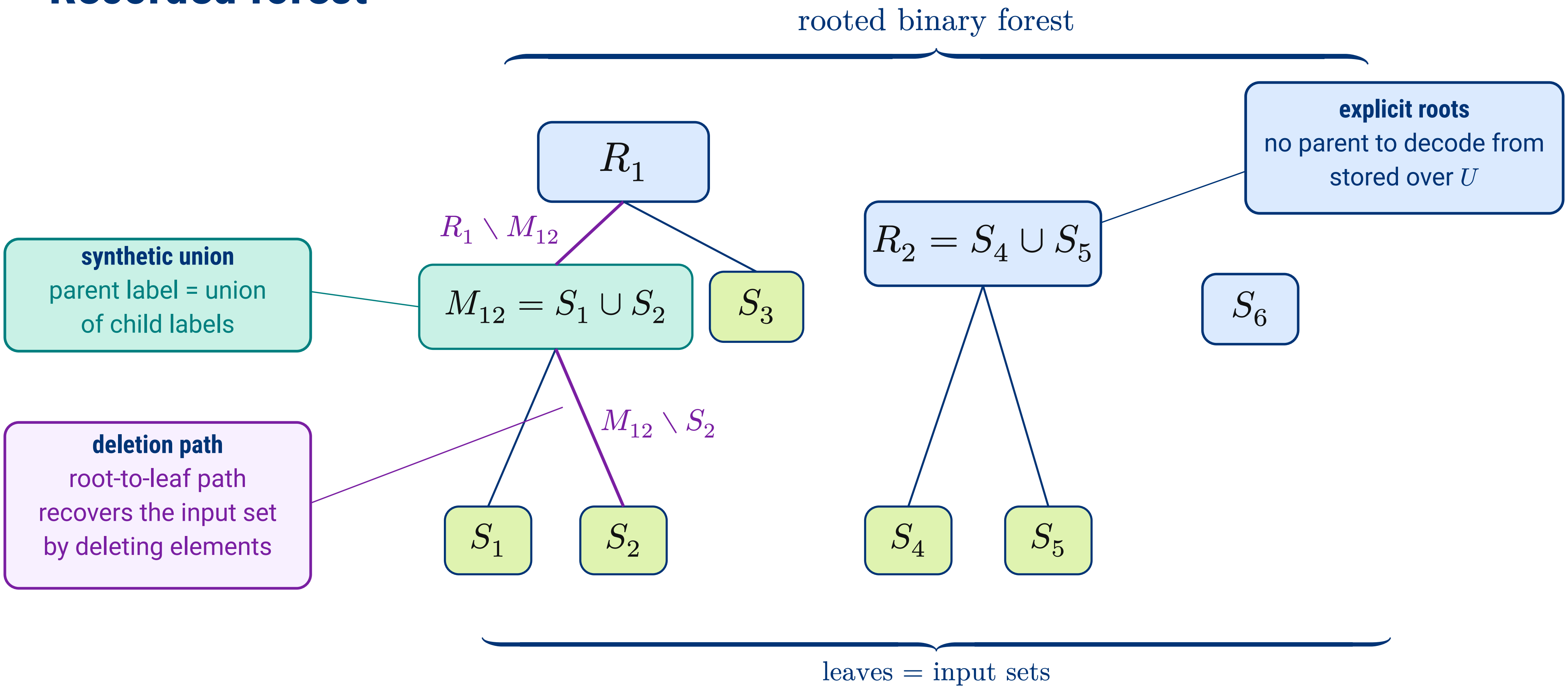




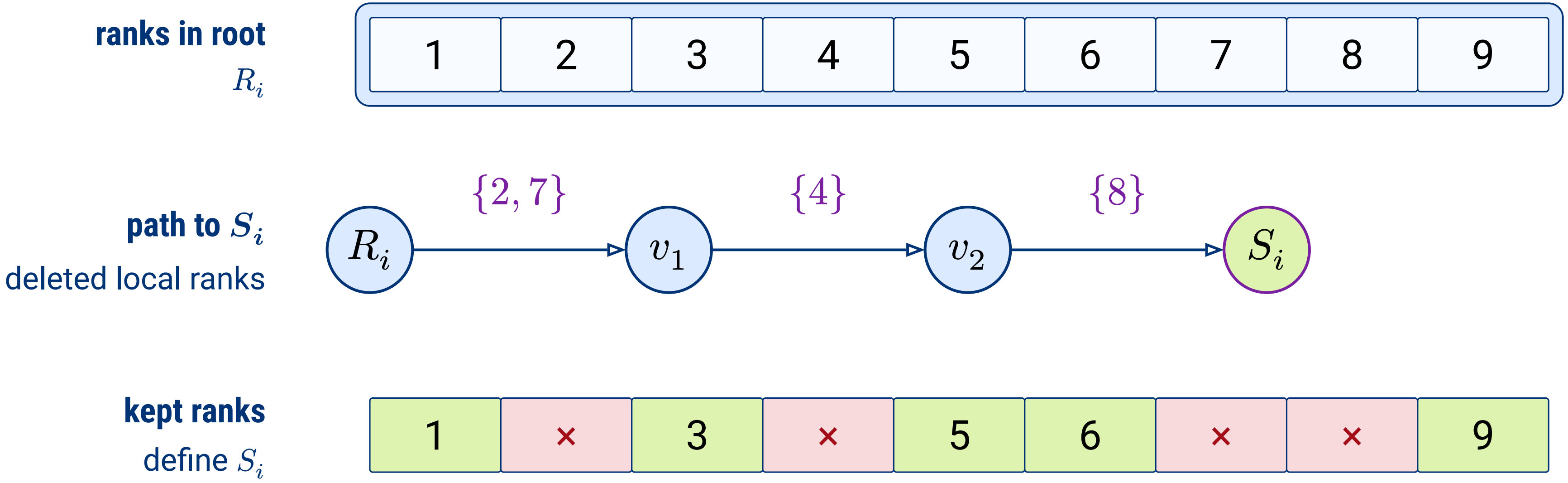
$$\Phi(F^*) = \min_t \Phi(F_t) \le \Phi(F_0) = H_{\mathrm{wc}}(\mathcal{S}).$$

Because F_0 has no merges, the independent encoding is always among the recorded candidates.

Recorded forest



Survivor queries



$D(v_i) = \{2,4,7,8\}$ and $S_i = \{x_j : j \in [1..9] \setminus D(v_i)\} = \{x_1, x_3, x_5, x_6, x_9\}$

$\mathbf{member}(S_i, x)$

$j = \mathrm{rank}_{R_i}(x)$

true iff rank j survives

$\mathbf{rank}(S_i, x)$

$j = \mathrm{rank}_{R_i}(x)$

$j - |D(v_i) \cap [1..j]|$

$\mathbf{access}(S_i, q)$

select the q-th survivor rank j

return $\mathrm{select}_{R_i}(j)$

Query times

A query on S_i becomes a **path query** from its component root R_i to its leaf, over local deletion ranks indexed by the framework of He, Munro, and Zhou (2014, 2016).

Theorem (SUM query times)

In the word RAM model with word size $\omega = \Theta(\lg n)$, for nonconstant $|R_i|$, the representation supports:

Operation	*Question on the path*	*Time*
$\mathrm{member}(S_i, x)$	*was the rank of x deleted?*	$O(\lg \lg_\omega \lvert R_i \rvert)$
$\mathrm{rank}(S_i, x)$	*how many deleted ranks lie before x?*	$O(\lg_\omega \lvert R_i \rvert)$
$\mathrm{access}(S_i, j)$	*which rank is the j-th survivor?*	$O\left(\frac{\lg \lvert R_i \rvert}{\lg \lg \lvert R_i \rvert}\right)$
predecessor, successor	*rank followed by access*	*same as access*

The parameter is the **component root size** $|R_i|$, not the universe size u.

Queryable storage

Theorem (Representation space)

Each component root R is stored directly as a rank/select dictionary over U.

$$\sum_{R \ \text{root}} \left(\lg \binom{u}{|R|} + o(u) + O(\lg \lg u) \right) \ \ \text{bits}$$

The query overlay stores the input directory, the deletion forest, and the HMZ path-query indexes. Here m is the number of input sets.

$$O\left(m + \sum_{c \ \text{internal}} |P_{\text{left}(c)} \triangle P_{\text{right}(c)}| \right) \ \ \text{words}$$

These data support the five selected-set queries.

Membership patterns

After building a queryable hierarchy, we can ask what any representation is trying to approach. **Ignore references** for a moment and describe the collection element by element.

Definition (Membership pattern)

$$\sigma(x) = \{i \in [m] : x \in S_i\}.$$

Equal patterns identify elements that no input set can distinguish. For each pattern $\tau \subseteq [m]$, collect all such elements in one region.

$$A_\tau = \{x \in U : \sigma(x) = \tau\}.$$

The regions A_τ partition U. Their sizes count how many times the collection repeats each membership pattern, independently of any chosen hierarchy.

Atom bound

Once the pattern regions and their sizes are fixed, the remaining choice is how the labelled elements of U are assigned to those regions.

Definition (Realised patterns and atom sizes)

$$R(\mathcal{S}) = \{\tau \subseteq [m] : A_\tau \neq \emptyset\}, \quad c_\tau = |A_\tau| \text{ for } \tau \in R(\mathcal{S}).$$

The assignment must respect that exactly c_τ elements receive each realised pattern τ.

Proposition (Atom bound)

With this side information, every lossless representation needs at least

$$L^*(\mathcal{S}) = \lg \binom{u}{(c_\tau)_{\tau \in R(\mathcal{S})}} \text{ bits.}$$

Global code

The lower bound is attainable as a global enumerative code.

Proposition (Achievability of L^*)

Given $R(\mathcal{S})$ and $(c_\tau)_{\tau \in R(\mathcal{S})}$, the collection can be encoded in exactly $\lceil L^*(\mathcal{S}) \rceil$ bits.

Order $U = \{x_1, \ldots, x_u\}$. The encoder maps the collection to the word of membership patterns along this order. Its alphabet is $R(\mathcal{S})$, and c_τ fixes how many times each symbol τ appears.

$$\pi(\mathcal{S}) = (\sigma(x_1), \ldots, \sigma(x_u)) \in R(\mathcal{S})^u \quad |\{j : \sigma(x_j) = \tau\}| = c_\tau$$

The code is optimal because it identifies this word among all words with the same counts. It remains global, while SUM is queryable because it exposes local root-to-leaf paths.

Compressed Representations of Set Collections

Thank you for listening!

Any question?